S.E.P. **S.E.S.** **TecNM**

Instituto Tecnológico de Aguascalientes

Tecnologías de la Información y la Comunicación

ACTIVIDAD

1_Práctica NFR24L01 – HandShake

Materia

Tecnologías Inalámbricas

Maestro: RICARDO ALEJANDRO RODRIGUEZ JIMENEZ

Alumnos:

Casas Demetrio Jorge Alberto

Zacarias Garcia Jason Alexander

Fecha: 5 de marzo de 2026

Contenido

Introducción 3

Contenido 4

Link de video 4

Link de GitHub 4

Descripciones 4

Código Arduino 6

ESP32-WROOM-32 6

ESP32S3 7

Diagrama de Conexiones 9

ESP32-WROOM-32 9

ESP32S3 9

Materiales 10

Evidencias 11

..... 11

Conclusión 12

Introducción

Esta práctica se centra en la implementación de un sistema de comunicación por radiofrecuencia (RF) utilizando el transceptor nRF24L01 en conjunto con microcontroladores de alto rendimiento de la familia ESP32.

El nRF24L01 es un módulo que opera en la banda de frecuencia ISM (Industrial, Scientific, and Medical) de 2.4 GHz, utilizando modulación GFSK para el envío de datos. Su bajo costo, reducido consumo energético y capacidad para manejar múltiples canales de comunicación lo convierten en una solución ideal para redes de sensores inalámbricos donde el Wi-Fi o el Bluetooth podrían ser excesivos en términos de consumo de recursos.

Por otro lado, el uso del ESP32-S3 / WROOM32 proporciona una plataforma robusta con capacidades de procesamiento de doble núcleo y una gestión eficiente de los buses de comunicación serial, como el SPI (Serial Peripheral Interface), el cual es el protocolo encargado de la transferencia de datos entre el microcontrolador y el módulo de radio.

Contenido

Link de video

https://youtu.be/ykh4sr-4PxM

Link de GitHub

https://github.com/JasonMakana/Practica-HandShake

Descripciones

ESP32-S3: Es una versión evolucionada de la familia ESP32, diseñada específicamente para aplicaciones de Inteligencia Artificial y Machine Learning en el borde (Edge Computing).

- Procesador: XTensa® Dual-Core LX7 de 32 bits, con una frecuencia de hasta 240 MHz.
- Características Clave: Incluye instrucciones vectoriales que aceleran el procesamiento de señales y cálculos de IA.
- Conectividad: Wi-Fi de 2.4 GHz y Bluetooth 5 (LE) con mayor alcance.
- Pines: Cuenta con 45 GPIOs programables y soporte para interfaces USB nativas, lo que facilita su programación sin necesidad de chips convertidores externos.

ESP32-WROOM-32: Es el módulo estándar y más popular de la familia ESP32, conocido por su versatilidad y bajo costo en proyectos de IoT general.

- Procesador: Dual-Core Tensilica Xtensa LX6 de 32 bits.
- Conectividad: Wi-Fi 802.11 b/g/n y Bluetooth de modo dual (Clásico y LE).
- Uso en la práctica: Es ideal para actuar como nodo receptor o transmisor debido a su robusto manejo del protocolo SPI, permitiendo una integración sencilla con periféricos externos como sensores y módulos de radio.

Módulo nRF24L01: Es un transceptor de radiofrecuencia (RF) de baja potencia que opera en la banda global de 2.4 GHz.

- Funcionamiento: Utiliza el bus SPI para comunicarse con el microcontrolador. Es capaz de manejar una red de hasta 6 dispositivos enviando datos simultáneamente a un solo receptor.
- Eficiencia: Su consumo de corriente es extremadamente bajo (aprox. 12mA en transmisión), lo que lo hace perfecto para dispositivos que funcionan con baterías.

Diferenciación: nRF24L01 vs. nRF24L01 con PA+LNA

nRF24L01 Estándar (Sin antena externa)

- Antena: Utiliza una antena de traza (dibujada en el PCB).
- Alcance: Limitado, generalmente de 10 a 30 metros en interiores.
- Uso: Ideal para proyectos de corto alcance, como controles remotos o sensores en la misma habitación.

nRF24L01 con PA + LNA (Con antena externa)

- PA (Power Amplifier): Incluye un amplificador que aumenta la potencia de la señal transmitida.
- LNA (Low Noise Amplifier): Un amplificador de bajo ruido que ayuda a captar señales muy débiles del aire.
- Antena: Viene con un conector SMA y una antena externa (similar a la de un router).
- Alcance: Puede llegar hasta 800 - 1000 metros en línea de visión directa.
- Consumo: Requiere más corriente, por lo que es casi obligatorio usar un capacitor de filtrado para evitar caídas de voltaje.

Código Arduino

ESP32-WROOM-32

```
//NRF24L01 PA/LNA Wireless LED Control
//Transmitter code#include <SPI. h>

#include <SPI.h>
#include <RF24.h>
#include <nRF24L01.h>
#include <Adafruit_NeoPixel.h>

#define PIN_RGB 48  // Pin del sensor led
RF24 radio(2, 5);   // CE, CSN
const byte address[6] = "12321";
int datoRecibido = 0;

Adafruit_NeoPixel pixels(1, PIN_RGB, NEO_GRB + NEO_KHZ800);
/*
void setColor(uint8_t r, uint8_t g, uint8_t b) {
  pixels.setPixelColor(0, pixels.Color(r, g, b));
  pixels.show();
}
*/
void setup() {
  Serial.begin(115200);
  //pixels.begin();
  //pixels.setBrightness(30);

  SPI.begin(18, 19, 23, 5);

  Serial.println("\nCircuito Receptor");
  //radio.begin();

  if (!radio.begin()) {
    Serial.println("No se detecta el nRF24L01");
    while (1);
  }

  radio.openReadingPipe(1, address);
  radio.setPALevel(RF24_PA_LOW);  // radio. setPALevel
  radio.startListening();
}
```

```
void loop() {
  if (radio.available()) {
    radio.read(&datoRecibido, sizeof(datoRecibido));

    Serial.print("\nEstado recibio: ");
    Serial.println(datoRecibido);

    if (datoRecibido > 0) {
      Serial.println("Dato enviado");
    } else {
      Serial.println("Sin señal");
    }
    delay(1000);
  }
}
```

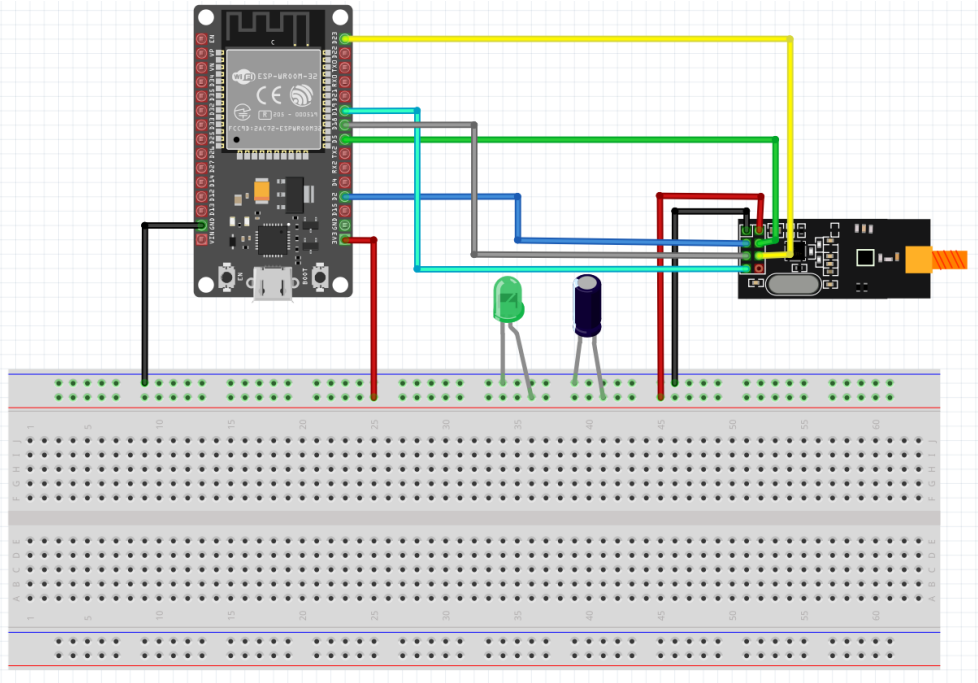
ESP32S3

```
//NRF24L01 PA/LNA Wireless LED Control
//Receiver code


#include <RF24.h>
#include <nRF24L01.h>
#include <printf.h>

#define boton 4
int contador = 0;
RF24 radio(4, 5);  // CE, CSN
const byte address[6] = "12321";

void setup() {
  Serial.begin(115200);
  Serial.println("\nCircuito Transmisor");
  pinMode(boton, INPUT_PULLUP);

  SPI.begin(12, 13, 11, 5);  // SCK, MISO, MOSI, SS (CSN)

  radio.begin();

  if (!radio.begin()) {
    Serial.println("No se detecta el nRF24L01");
    while (1);
  }
```

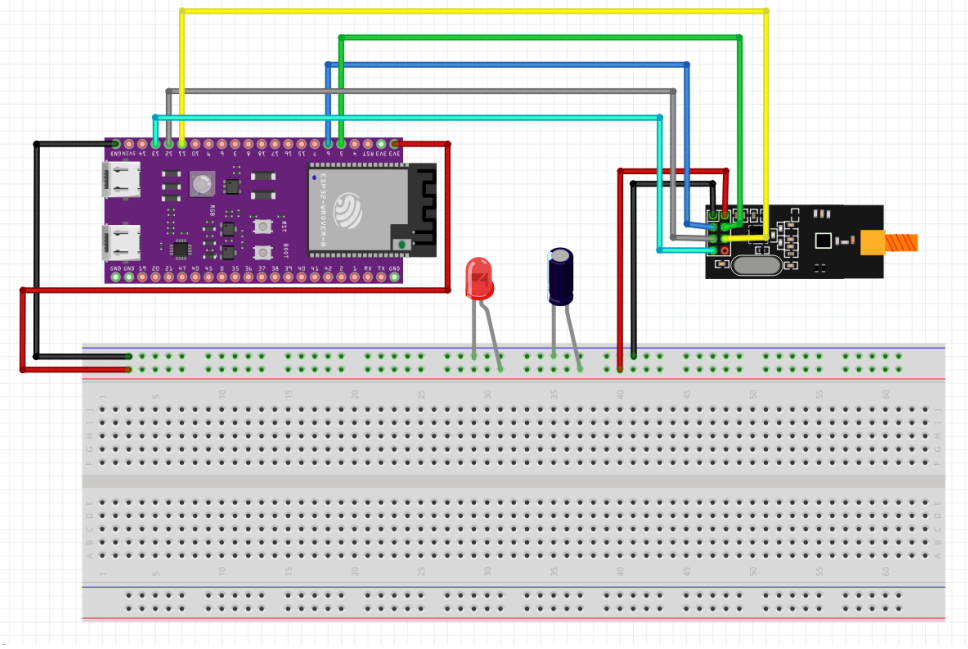
```
  radio.openWritingPipe(address);
  radio.setPALevel(RF24_PA_LOW);
  radio.stopListening();
}

void loop() {
  Serial.print("Enviando dato: ");
  Serial.println(contador);

  bool ok = radio.write(&contador, sizeof(contador));

  if (ok) {
    Serial.println(">> ¡Confirmado!");
  } else {
    Serial.println(">> Sin respuesta");
  }

  contador++;
  delay(1000); // Envía cada 1 segundo
}
```

Diagrama de Conexiones

ESP32-WROOM-32



ESP32S3



Materiales

Microcontrolador ESP32 (S3): Su función principal es gestionar la lógica de control, procesar los paquetes de datos y coordinar la comunicación con los módulos de radio. Actúa como el Emisor de la practica

Microcontrolador ESP32 (WROOM32): Su función principal es gestionar la lógica de control, procesar los paquetes de datos y coordinar la comunicación con los módulos de radio. Actúa como Receptor

Dos Módulos Transceptores nRF24L01+: Son los dispositivos encargados de la capa física de comunicación por radiofrecuencia. Operan en la banda libre de 2.4 GHz y convirte los datos digitales en señales electromagnéticas (y viceversa).

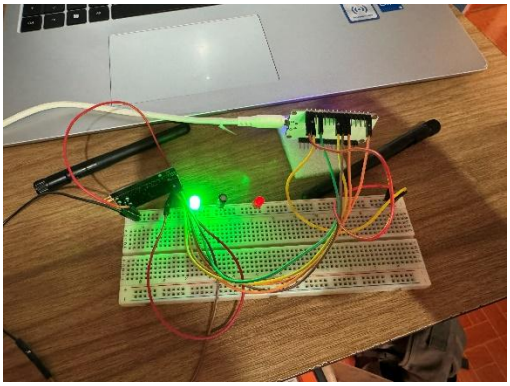
Cables de Conexión (Jumpers): Conductores tipo necesarios para establecer las conexiones entre los pines GPIO del ESP32 y los pines de interfaz del nRF24L01

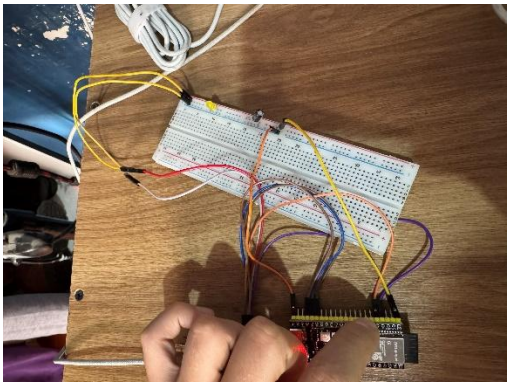
Dos Cables de Datos USB (Tipo C y Micro-USB): Suministran energía desde la computadora a las placas de desarrollo y permitir la carga del firmware, así como la apertura de la terminal serial para el monitoreo de la comunicación.

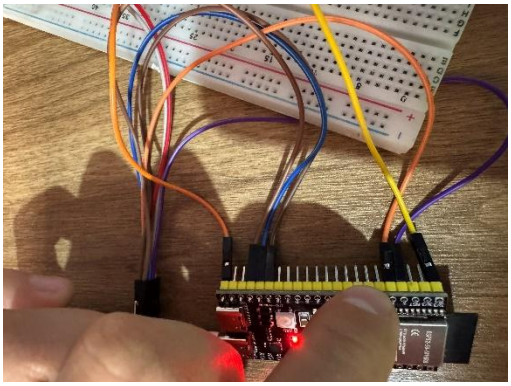
Capacitores Electrolíticos 10µF: Estabilizan la línea de alimentación de 3.3V de los módulos nRF24L01.

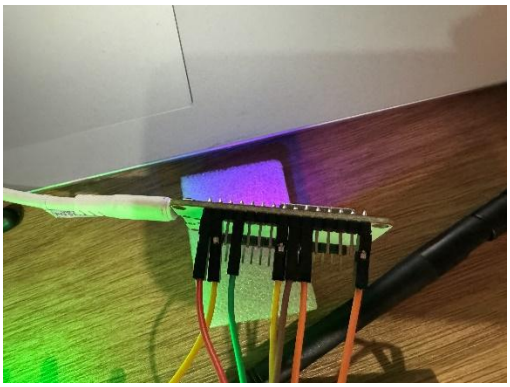
LED (Light-Emitting Diode): Utilizados para poder confirmar la continuidad de corriente.

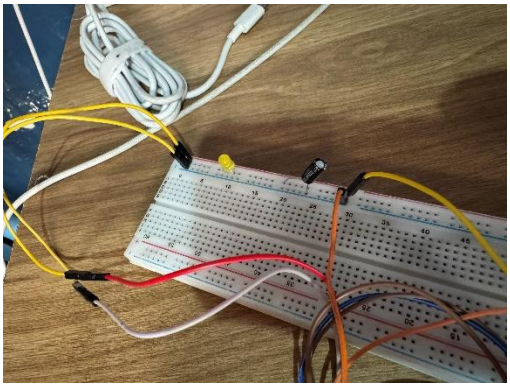
Evidencias

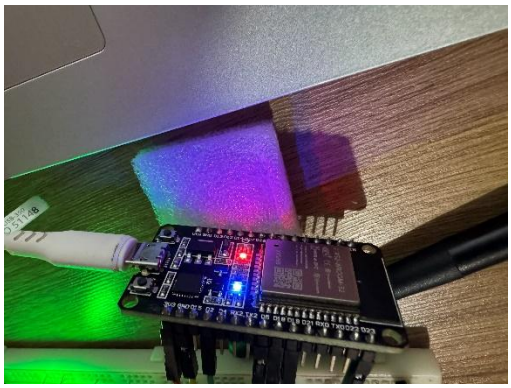


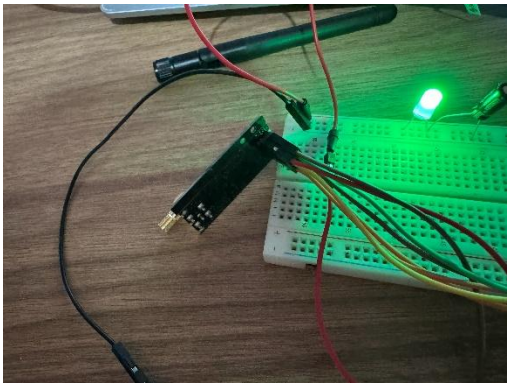












Conclusión

Jason Alexander Zacarias Garcia

En esta práctica comprendí que para lograr una buena comunicación entre dispositivos no basta con que el programa esté bien hecho, y también que es muy importante conectar correctamente los componentes físico (se aprendió a la mala). Aprendí que los pines pueden cambiar dependiendo del modelo de ESP32 que se utilice, por lo que es necesario revisar bien las conexiones. Además, entendí que el módulo nRF24L01 necesita una alimentación estable para funcionar correctamente y que ambos dispositivos deben estar bien configurados para que los datos se envíen y reciban sin problemas.

Jorge Alberto Casas Demetrio

En esta práctica aprendí a organizar la comunicación entre dispositivos, donde uno envía la información y el otro la recibe. También entendí la importancia de revisar si los datos realmente se están enviando y llegando correctamente. Para esto se utilizó el monitor serial, que nos ayudó a ver lo que estaba pasando y a detectar posibles errores o el mal funcionamiento. Todo este proceso me ayudó a mejorar mi forma de analizar problemas y buscar soluciones cuando algo no funciona como debería.